- *Singleton*—You always get the same instance of the service, regardless of scope.

NOTE These concepts align well with most other DI containers, but the terminology may differ. If you're familiar with a third-party DI container, be sure you understand how the lifetime concepts align with the built-in ASP.NET Core DI container.

To illustrate the behavior of each lifetime, I use a simple example in this section. Suppose that you have `DataContext`, which has a connection to a database, as shown in listing 9.10. It has a single property, `RowCount`, which represents the number of rows in the `Users` table of a database. For the purposes of this example, we emulate calling the database by setting the number of rows randomly when the `DataContext` object is created, so you always get the same value every time you call `RowCount` on a given `DataContext` instance. Different instances of `DataContext` return different `RowCount` values.

Listing 9.10 `DataContext` generating a random `RowCount` on creation

```
class DataContext
{
    public int RowCount { get; }                ❶
        = Random.Shared.Next(1, 1_000_000_000); ❷
}
```

❶ The property is read-only, so it always returns the same value.

❷ Generates a random number between 1 and 1,000,000,000

You also have a `Repository` class that has a dependency on the `DataContext`, as shown in the next listing. *It* also exposes a `RowCount` property, but this property delegates the call to its instance of